UNIDAD 2
ACTIVIDAD PRÁCTICA
MEDIR EL FILTRADO Y LA REPETIBILIDAD DE LOS DETECTORES

U-tad
CENTRO UNIVERSITARIO
DE TECNOLOGÍA Y ARTE DIGITAL

Centro adscrito a
Universidad
Camilo José Cela

ÍNDICE

ORGANIZACIÓN DE LA ACTIVIDAD PRÁCTICA 1

DESCRIPCIÓN DE LA ACTIVIDAD PRÁCTICA 1

EJERCICIO 1. REJILLA DE DENOISING CON MÉTRICAS OBJETIVAS 1

EJERCICIO 2. LECTURA CUANTITATIVA DEL ESPECTRO 1

EJERCICIO 3. CANNY CON UMBRALES ELEGIDOS POR EVIDENCIA 1

EJERCICIO 4. REPETIBILIDAD DE KEYPOINTS BAJO TRANSFORMACIÓN CONOCIDA 1

EJERCICIO 5. INFORME DE HALLAZGOS 1

ENTREGABLES VERIFICABLES 1

RÚBRICA DE CORRECCIÓN 1

CÓMO REALIZAR LA ENTREGA 1

ORGANIZACIÓN DE LA ACTIVIDAD PRÁCTICA

Nombre de la práctica	Medir el filtrado y la repetibilidad de los detectores
Tipo de tarea	Individual
Entregables	Archivo único comprimido (.zip)
Peso en la evaluación	Según la guía docente de la asignatura

DESCRIPCIÓN DE LA ACTIVIDAD PRÁCTICA

La Unidad 2 nos ha dado una caja de herramientas amplia: convolución, lectura en frecuencia, filtros lineales y no lineales, detección de bordes y detección de estructuras de interés. El riesgo de una caja de herramientas amplia es evidente: acabamos probando cosas hasta que algo parece funcionar, sin llegar a saber por qué funciona ni cuándo dejaría de hacerlo.

Esta práctica adopta una postura distinta. En lugar de aplicar filtros y comentar el resultado, vamos a montar un banco de pruebas. Un banco de pruebas es un dispositivo que, dado un problema con solución conocida, permite puntuar objetivamente cualquier candidato. Con él podremos responder preguntas del tipo "¿qué filtro limpia mejor el ruido de sal y pimienta al 30 %?" con una cifra en lugar de con una opinión, y podremos comprobar afirmaciones de la unidad en vez de aceptarlas por autoridad.

Hay una segunda idea que atraviesa toda la práctica: si queremos medir, necesitamos una referencia. Por eso en varios ejercicios construiremos escenas sintéticas o aplicaremos transformaciones conocidas. Cuando sabemos exactamente dónde están los bordes o exactamente cómo se ha movido la imagen, podemos calcular el acierto de un detector. Sobre una fotografía cualquiera, no.

Lo que cambia respecto a lo habitual. Ningún parámetro de esta práctica se elige a ojo. Cada umbral, cada sigma y cada tamaño de kernel tiene que salir de un barrido con un criterio declarado. Si en la memoria aparece un valor sin la tabla o la curva que lo justifica, ese valor no cuenta.

Material de partida

Necesitas tres imágenes en escala de grises, o convertidas a escala de grises por código:

1. Una imagen con textura fina y detalle abundante, del tipo de una fachada, un tejido o vegetación.
2. Una imagen con regiones amplias y uniformes separadas por bordes marcados, del tipo de un objeto sobre fondo liso o una señal de tráfico.
3. Una imagen con ruido real apreciable, tomada con poca luz o con sensibilidad alta. Si no dispones de ninguna, indícalo y trabaja con las dos anteriores más ruido sintético.

Además, generarás por código una escena sintética que servirá de patrón de referencia en el Ejercicio 3.

EJERCICIO 1. REJILLA DE DENOISING CON MÉTRICAS OBJETIVAS

La unidad afirma que la mediana funciona mejor contra el ruido impulsivo y que el bilateral preserva bordes mejor que un promedio. Vamos a comprobarlo, y de paso a cuantificar cuánto mejor y hasta qué nivel de ruido se sostiene la afirmación.

Por qué necesitamos dos métricas y no una

Para comparar un resultado con la imagen limpia de partida hace falta una medida de parecido. La más inmediata es la relación señal a ruido de pico (PSNR), que se deriva del error cuadrático medio entre ambas imágenes. Es fácil de calcular y de interpretar, pero tiene un defecto conocido: penaliza por igual un error repartido de forma imperceptible y un error concentrado en un borde, que sí se nota.

Por eso añadimos una segunda medida, el índice de similitud estructural (SSIM), que compara luminancia, contraste y estructura local en lugar de restar píxel a píxel. Un filtro que emborrona mucho puede salir razonablemente bien en PSNR y claramente mal en SSIM. Justamente esa discrepancia es lo que queremos observar, porque es la traducción numérica del compromiso entre eliminar ruido y conservar detalle del que habla la unidad.

La rejilla

Parte de tus imágenes limpias. Inyecta ruido de forma controlada y reproducible:

```
import numpy as np

RNG = np.random.default_rng(20260101)   # semilla fija: mismo ruido en cada ejecución

def ruido_sal_y_pimienta(img, p):
    """Sustituye una fracción p de los píxeles por 0 o 255. Es ruido
    impulsivo: pocos píxeles, pero completamente equivocados."""
    out = img.copy()
    n = int(p * img.size)
    filas = RNG.integers(0, img.shape[0], n)
    cols = RNG.integers(0, img.shape[1], n)
    out[filas, cols] = RNG.choice([0, 255], size=n)
    return out

def ruido_gaussiano(img, sigma):
    """Suma una perturbación gaussiana a todos los píxeles. Es ruido
    de fondo: afecta a toda la imagen con magnitud pequeña."""
    perturbacion = RNG.normal(0, sigma, img.shape)
    return np.clip(img.astype(np.float32) + perturbacion, 0, 255).astype(np.uint8)
```

Recorre entonces esta rejilla completa sobre cada imagen:

Eje	Valores
Tipo y nivel de ruido	Sal y pimienta al 5 %, 10 % y 30 %. Gaussiano con σ de 10, 20 y 40. Seis condiciones.
Filtros a evaluar	Media 3×3 · Gaussiano con σ de 1,0 y 2,0 · Mediana 3×3 y 5×5 · Mediana híbrida · Mediana truncada · Bilateral con dos configuraciones distintas de σ espacial y σ de rango. Diez filtros.
Salida por celda	PSNR y SSIM frente a la imagen limpia, más el tiempo de ejecución en milisegundos.

Son 60 combinaciones por imagen. La mediana híbrida y la mediana truncada tendrás que implementarlas: no vienen en OpenCV, y escribirlas es la mejor forma de entender en qué se diferencian de la mediana clásica.

El fichero de resultados

```
outputs/resultados_denoising.csv

imagen,tipo_ruido,nivel,filtro,parametros,psnr,ssim,tiempo_ms
fachada.png,sal_pimienta,0.30,mediana,k=3,26.41,0.812,4.2
fachada.png,sal_pimienta,0.30,media,k=3,19.87,0.534,1.1
fachada.png,gaussiano,20,bilateral,d=9;sc=75;ss=75,28.03,0.871,31.6
...
```

Qué tienes que responder

1. Para cada tipo de ruido y cada nivel, cuál es el filtro ganador en PSNR y cuál en SSIM. Si no coinciden, explica qué está midiendo cada métrica en ese caso concreto.
2. Si la ventaja de la mediana frente a la media sobre ruido impulsivo se mantiene en los tres niveles, y si crece o se reduce al aumentar el porcentaje de ruido.
3. Qué le ocurre a la mediana cuando se aplica sobre ruido gaussiano, y por qué su comportamiento cambia respecto al caso impulsivo.
4. Cuánto cuesta en tiempo el bilateral frente a un gaussiano de suavizado comparable, y si esa diferencia compensa según tus cifras de SSIM.
5. Un caso en el que el filtro con mejor cifra no sea el que tú elegirías visualmente. Si no lo encuentras, dilo también.

EJERCICIO 2. LECTURA CUANTITATIVA DEL ESPECTRO

En la unidad vimos que el filtro de caja produce lóbulos laterales en frecuencia mientras que el gaussiano atenúa de forma suave. Mirar dos espectros uno al lado del otro sugiere esa diferencia, pero no la demuestra. En este ejercicio la convertimos en una curva y en un número.

El perfil radial

El espectro de magnitud de una imagen es bidimensional, lo que dificulta compararlo. Como los filtros que nos ocupan son aproximadamente isótropos, podemos reducirlo a una curva promediando la magnitud sobre anillos concéntricos alrededor del centro. El resultado es el perfil radial: cuánta energía queda en cada frecuencia, de la más baja a la más alta.

```
def perfil_radial(espectro):
    """Promedia la magnitud del espectro sobre anillos concéntricos.
    Convierte un espectro 2D en una curva 1D de energía por frecuencia,
    mucho más fácil de comparar entre filtros."""
    alto, ancho = espectro.shape
    cy, cx = alto // 2, ancho // 2
    y, x = np.indices((alto, ancho))
    radio = np.sqrt((x - cx) ** 2 + (y - cy) ** 2).astype(int)
    suma = np.bincount(radio.ravel(), weights=espectro.ravel())
    cuenta = np.bincount(radio.ravel())
    return suma / np.maximum(cuenta, 1)
```

Qué medir

Para una de tus imágenes, calcula el perfil radial del espectro en cinco casos: la imagen original, filtrada con caja de 3×3, 5×5 y 9×9, y filtrada con gaussiano de σ igual a 0,8, 1,6 y 3,0. Normaliza cada perfil por su valor en frecuencia cero para poder superponerlos.

Sobre esas curvas, extrae dos cifras por filtro:

1. La frecuencia de corte, definida como el radio al que el perfil normalizado cae por debajo de 0,5. Es una medida directa de cuánto suaviza el filtro.
2. El número de veces que el perfil vuelve a subir después de haber bajado. En un gaussiano debería ser cero o casi cero; en el filtro de caja, no.

Ese segundo número es la traducción cuantitativa de los lóbulos laterales. Con él ya no hace falta discutir si se ven o no se ven en el espectro.

La pregunta que cierra el ejercicio

Empareja cada filtro de caja con el gaussiano que tenga la frecuencia de corte más parecida, según tus propias cifras. Para cada pareja, compara el resultado visual y el SSIM frente a la original. A igualdad de suavizado medido en frecuencia, ¿cuál conserva mejor la imagen? Responde con la cifra de SSIM, no con la impresión.

EJERCICIO 3. CANNY CON UMBRALES ELEGIDOS POR EVIDENCIA

La elección de los dos umbrales de Canny es el ejemplo perfecto de decisión que suele tomarse a ojo. En la unidad se propuso un criterio práctico basado en percentiles, y en este ejercicio vamos a comprobar si ese criterio es bueno, comparándolo con la mejor elección posible sobre un problema del que conocemos la respuesta.

Paso 1: fabricar la referencia

Genera por código una imagen sintética de 512×512 con formas geométricas de bordes conocidos: rectángulos, círculos y algunos segmentos, con distintos niveles de contraste entre figura y fondo. Al dibujarlas tú, sabes exactamente dónde está cada borde, así que puedes generar simultáneamente la imagen y su mapa de bordes de referencia.

Añade después ruido gaussiano de σ igual a 15 sobre la imagen, pero no sobre la referencia. Ahora tienes un problema realista con solución conocida.

> ***Por qué sintética y no una foto. Sobre una fotografía no existe un mapa de bordes verdadero: dos personas marcarían contornos distintos. Sin referencia no hay forma de puntuar objetivamente un detector, y volveríamos a las impresiones. La escena sintética es el precio que pagamos por poder medir.***

Paso 2: puntuar una detección

Para comparar un mapa de bordes con la referencia, cuenta aciertos con tolerancia espacial: un píxel detectado se considera correcto si hay algún píxel de borde de referencia a distancia menor o igual que 2. Con esa definición puedes calcular precisión, exhaustividad y F1 sobre los píxeles de borde. La tolerancia es necesaria porque un borde bien detectado pero desplazado un píxel no es un error real.

Paso 3: el barrido

Recorre una rejilla de umbrales. Para el umbral alto, prueba los percentiles 70, 75, 80, 85, 90 y 95 de las magnitudes de gradiente no nulas. Para el bajo, prueba fracciones de 0,3, 0,4, 0,5 y 0,6 del alto. Son 24 combinaciones. Para cada una, registra precisión, exhaustividad, F1 y el número total de píxeles marcados como borde.

```
outputs/barrido_canny.csv

percentil_alto,fraccion_baja,umbral_alto,umbral_bajo,precision,recall,f1,px_borde
85,0.5,142,71,0.874,0.791,0.830,18342
90,0.5,167,83,0.921,0.702,0.797,15104
...
```

Paso 4: la conclusión

1. Localiza la combinación con mejor F1 y anota sus valores.
2. Compárala con la que propone el criterio de la unidad, es decir, percentil 85 para el umbral alto y su mitad para el bajo. ¿Cuánto F1 se pierde por usar la regla general en lugar del óptimo de tu escena?
3. Dibuja la curva de precisión frente a exhaustividad recorriendo el percentil alto. Explica en qué dirección se mueve cada magnitud y por qué.

4. Aplica ahora tanto el óptimo hallado como la regla de la unidad a tus imágenes reales. Como ahí no hay referencia, no puedes puntuar, pero sí describir qué diferencias se observan y si los umbrales óptimos de la escena sintética transfieren bien.

Este último punto es el más importante de la práctica en cuanto a criterio profesional: un parámetro ajustado sobre un caso concreto no tiene por qué servir para otro, y comprobar hasta dónde transfiere es parte del trabajo.

EJERCICIO 4. REPETIBILIDAD DE KEYPOINTS BAJO TRANSFORMACIÓN CONOCIDA

Un detector de puntos clave sirve para volver a encontrar los mismos puntos en otra vista de la escena. Esa es literalmente su función. Sin embargo, casi nunca se mide: lo habitual es dibujar los círculos sobre la imagen, comprobar que caen en sitios plausibles y darlo por bueno.

En este ejercicio medimos lo que importa. La idea es sencilla: transformamos la imagen con una transformación que conocemos exactamente, detectamos puntos en ambas versiones, proyectamos los de la primera a la segunda usando la transformación y contamos cuántos tienen un vecino cercano. Esa fracción es la tasa de repetibilidad.

El montaje

```
import cv2
import numpy as np

# Nota sobre la API: el detector se construye con
# cv2.FastFeatureDetector_create(threshold=..., nonmaxSuppression=True)
# Otras grafías del nombre no existen y provocan un AttributeError.
detector = cv2.FastFeatureDetector_create(threshold=25, nonmaxSuppression=True)

def repetibilidad(img, H, tol=3.0):
    """Fracción de keypoints de la imagen original que reaparecen, tras
    aplicar la homografía H, a menos de `tol` píxeles de algún keypoint
    detectado en la imagen transformada."""
    alto, ancho = img.shape[:2]
    img_t = cv2.warpPerspective(img, H, (ancho, alto))

    kp_orig = detector.detect(img, None)
    kp_trans = detector.detect(img_t, None)
    if not kp_orig or not kp_trans:
        return 0.0, len(kp_orig), len(kp_trans)

    # Proyectamos los puntos originales al sistema de la imagen transformada
    pts = np.array([k.pt for k in kp_orig], dtype=np.float32).reshape(-1, 1, 2)
    proyectados = cv2.perspectiveTransform(pts, H).reshape(-1, 2)
    destino = np.array([k.pt for k in kp_trans], dtype=np.float32)

    # Solo cuentan los puntos que siguen dentro del encuadre tras la
    # transformación: los que se salen no es que no se repitan, es que
    # ya no están en la imagen.
    dentro = ((proyectados[:, 0] >= 0) & (proyectados[:, 0] < ancho) &
              (proyectados[:, 1] >= 0) & (proyectados[:, 1] < alto))
    proyectados = proyectados[dentro]
    if len(proyectados) == 0:
        return 0.0, len(kp_orig), len(kp_trans)

    d = np.linalg.norm(proyectados[:, None, :] - destino[None, :, :], axis=2)
    aciertos = (d.min(axis=1) <= tol).sum()
    return aciertos / len(proyectados), len(kp_orig), len(kp_trans)
```

El detalle del recorte. La comprobación de que el punto proyectado sigue dentro del encuadre no es un tecnicismo: sin ella, una rotación de 40 grados parece hundir la repetibilidad cuando lo único que ocurre es que las esquinas se han salido de la imagen. Es el tipo de error que produce conclusiones falsas con código que ejecuta sin quejarse.

Los barridos

Barrido	Transformación	Valores
Rotación	Giro alrededor del centro de la imagen	0°, 10°, 20°, 30°, 45°, 60°, 90°
Escala	Reescalado uniforme alrededor del centro	0,5 · 0,7 · 0,85 · 1,0 · 1,25 · 1,5 · 2,0
Iluminación	Cambio de brillo aplicado como gamma, sin transformación geométrica	$\gamma \in \{0{,}6; 0{,}8; 1{,}0; 1{,}3; 1{,}7\}$

Ejecuta los tres barridos sobre al menos dos imágenes. En el barrido de iluminación la homografía es la identidad, de modo que la repetibilidad mide únicamente la sensibilidad del detector al cambio de intensidad.

Las tres preguntas

1. Con las cifras de repetibilidad frente a escala, ¿confirmas o refutas que FAST en su formulación básica sea invariante a escala? Da el valor concreto de repetibilidad a escala 0,5 y a escala 2,0 y contrástalo con el de escala 1,0.
2. La repetibilidad frente a rotación, ¿es plana o tiene estructura? Fíjate especialmente en 90°, donde la rejilla de píxeles vuelve a alinearse, y compáralo con 45°. Si aparece un patrón, explícalo a partir de cómo está construido el test del anillo.
3. Repite el barrido de escala con un detector que sí explora varias escalas, como ORB o SIFT si tu instalación los incluye. Compara ambas curvas en la misma gráfica. La diferencia entre ellas es la demostración empírica de por qué existen los detectores multiescala.

EJERCICIO 5. INFORME DE HALLAZGOS

Cierra la práctica con un documento breve, `memoria/hallazgos.md`, de una página. Contiene exactamente cinco afirmaciones sobre lo que has aprendido, y cada una debe cumplir tres condiciones:

1. Ser una afirmación concreta y comprobable, no una generalidad. "El bilateral preserva mejor los bordes" no vale; "el bilateral supera al gaussiano en SSIM en 0,09 puntos sobre ruido gaussiano de σ=20, con un coste 28 veces mayor en tiempo" sí.
2. Ir acompañada del fichero y la fila de la que sale la cifra.
3. Incluir el límite de validez: sobre qué imágenes y en qué condiciones lo has comprobado.

Al menos una de las cinco afirmaciones debe recoger algo que te haya sorprendido o que contradiga lo que esperabas antes de medir. Si todo ha salido como suponías, revisa los datos con más atención: es poco frecuente.

ENTREGABLES VERIFICABLES

Artefacto	Qué debe contener	Criterio afectado
outputs/resultados_denoising.csv	Las 60 combinaciones por imagen del Ejercicio 1, con PSNR, SSIM y tiempo.	Evidencia cuantitativa
outputs/perfiles_frecuencia.csv	Frecuencia de corte y número de rebotes de cada filtro del Ejercicio 2.	Evidencia cuantitativa
outputs/barrido_canny.csv	Las 24 combinaciones de umbrales con precisión, exhaustividad y F1.	Evidencia cuantitativa
outputs/repetibilidad.csv	Repetibilidad para cada transformación, valor y detector.	Evidencia cuantitativa
memoria/hallazgos.md	Las cinco afirmaciones con su cifra, su origen y su límite de validez.	Análisis
codigo/requirements.txt	Versiones fijadas. Comprueba que la versión de OpenCV declarada incluye los detectores que usas.	Reproducibilidad
Semilla fijada	El ruido debe ser idéntico en cada ejecución. Sin semilla, tus cifras no son verificables.	Reproducibilidad

Lista de comprobación antes de entregar

- Reinicia el kernel y ejecuta de principio a fin. Un fallo por un nombre de función mal escrito invalida el bloque entero, no solo esa línea.
- Comprueba que ninguna celda instala paquetes. Las dependencias van en requirements.txt.
- Verifica que las cuatro rejillas están completas. Una condición de ruido ausente o un valor de escala que falta se detectan contando filas del CSV.
- Confirma que cada cifra citada en la memoria aparece en alguno de los CSV.
- Revisa que las gráficas comparativas comparten escala en el eje vertical cuando se comparan entre sí.

RÚBRICA DE CORRECCIÓN

La rúbrica sigue los criterios listados a continuación, con distintos pesos sobre la nota final.

Criterios	Excelente	Satisfactorio	No satisfactorio	Insuficiente
Corrección técnica y completitud (3 puntos)	De 2,5 a 3 puntos. Las cuatro rejillas completas. Mediana híbrida y truncada implementadas. Perfil radial correcto. Escena sintética con su referencia de bordes generada por código. Repetibilidad con el recorte de puntos fuera de encuadre bien resuelto.	De 2 a 2,5 puntos. Flujo correcto con fallos menores: alguna condición de la rejilla ausente, o alguna de las medianas resuelta con una aproximación.	De 1,5 a 2 puntos. Omisiones relevantes: falta un ejercicio completo, la rejilla de ruido está a medias, o la repetibilidad se calcula sin proyectar por la transformación.	De 0 a 1,5 puntos. Faltan bloques centrales o el código no ejecuta de principio a fin.
Evidencia cuantitativa y análisis (4 puntos)	De 3,5 a 4 puntos. Todos los parámetros salen de un barrido con criterio declarado. Se cuantifica la pérdida de usar la regla general frente al óptimo. La invarianza de FAST a escala se confirma o refuta con cifras. Se identifica y explica al menos una discrepancia entre PSNR y SSIM.	De 2,5 a 3,5 puntos. Análisis correcto con lagunas: los barridos existen pero la elección final no se justifica del todo, o falta la comparación entre métricas.	De 2 a 2,5 puntos. Los CSV existen pero la memoria no los usa. Parámetros elegidos a ojo con la tabla presente pero sin leer.	De 0 a 2 puntos. Sin análisis, o afirmaciones que los propios datos entregados contradicen.
Calidad de figuras y comunicación (1,5 puntos)	De 1,25 a 1,5 puntos. Curvas con ejes rotulados y unidades, escalas compartidas entre paneles comparables, mapas de calor con barra de color, y pies que incluyen parámetros y la cifra relevante.	De 1 a 1,25 puntos. Presentación clara con alguna incoherencia menor en escalas o algún pie incompleto.	De 0,75 a 1 puntos. Figuras sin rotular o escalas inconsistentes que impiden la comparación.	De 0 a 0,75 puntos. Resultados difíciles de interpretar o figuras ausentes.
Reproducibilidad y estilo de código (1,5 puntos)	De 1,25 a 1,5 puntos. Semilla fijada para el ruido, código modular con funciones	De 1 a 1,25 puntos. Código limpio con pequeños atajos,	De 0,5 a 1 puntos. Ejecuta pero es monolítico, o el ruido no es	De 0 a 0,5 puntos. No reproducible: falla al ejecutar en

Criterios	Excelente	Satisfactorio	No satisfactorio	Insuficiente
	reutilizables, rutas relativas, requirements con versiones y ejecución verificada en limpio.	o requirements sin versiones.	reproducible por falta de semilla.	limpio o los resultados no se pueden regenerar.

CÓMO REALIZAR LA ENTREGA

Archivo único comprimido (.zip) con nombre: **U2_Practica_Apellidos_Nombre.zip**

Estructura interna obligatoria:

```
U2_Practica_Apellidos_Nombre/
├─ memoria/
│   ├─ informe.pdf
│   └─ hallazgos.md
├─ codigo/
│   ├─ banco_pruebas.ipynb   (o .py)
│   └─ requirements.txt
├─ data/
│   └─ (las tres imágenes originales)
└─ outputs/
    ├─ resultados_denoising.csv
    ├─ perfiles_frecuencia.csv
    ├─ barrido_canny.csv
    ├─ repetibilidad.csv
    └─ (figuras generadas)
```

Memoria: PDF (no Word). Es el formato exigido; una entrega en otro formato se penaliza en el criterio de comunicación.

Código: Jupyter Notebook (.ipynb) o script (.py) ejecutable de principio a fin, sin instalar dependencias dentro de las celdas.

Figuras: PNG o JPG con nombres autoexplicativos, generadas por el propio código.

Resultados: los ficheros CSV que se piden en cada ejercicio deben venir generados por el código, no escritos a mano.

Límites prácticos: tamaño total ≤ 200 MB. No incluyas pesos de modelos salvo que se pidan explícitamente.